

MODULE 2

Detailed Day-by-Day Study Guide

Python Intermediate · AWS Cloud Practitioner

August 3 – August 31, 2026 | 25 Study Days | 3–4 Hours/Day Projects: Expense Tracker (Aug 10) · Contact Book (Aug 17) · Weather API App (Aug 24) S3 Static Website (Aug 28) · EC2 Web Server (Aug 31)

AWS Cloud Practitioner Exam Target: August 31, 2026

TIME	ACTIVITY
8:00 – 9:30 AM	Python intermediate lesson + notes
9:30 – 9:45 AM	Break
9:45 – 11:15 AM	Python project work / coding lab
11:15 – 11:45 AM	AWS Cloud Practitioner study (Stephane Maarek or Andrew Brown)
Evening (opt.)	AWS flashcards, practice questions, GitHub commit

DAY 1 · Monday, August 3

File I/O — Reading and Writing Files

Python talks to the real world through files. This is used in every real project.

Lesson: Opening and Reading Files

The built-in `open()` function gives you access to files on disk. Always use the **with** statement — it automatically closes the file when done, even if an error occurs.

```
# Open and read an entire file with open('notes.txt', 'r') as f: content = f.read() # entire
file as one string print(content) # Read line by line (memory-efficient for large files) with
open('notes.txt', 'r') as f: for line in f: print(line.strip()) # .strip() removes the trailing
newline # Read all lines into a list with open('notes.txt', 'r') as f: lines = f.readlines() #
['line1\n', 'line2\n', ...] print(lines[0]) # first line
```

Lesson: Writing Files

```
# Write to a file (creates it if it doesn't exist, OVERWRITES if it does) with
open('output.txt', 'w') as f: f.write('Hello, cloud engineer!\n') f.write('This is line 2.\n')
# Append to a file (adds to the end — does NOT overwrite) with open('output.txt', 'a') as f:
f.write('This line is appended.\n') # File modes: # 'r' read (default) — file must exist # 'w'
write — creates or OVERWRITES # 'a' append — creates or ADDS to end # 'x' exclusive create —
fails if file already exists # 'r+' read and write — file must exist
```

Lesson: os Module Basics

```
import os print(os.getcwd()) # print current working directory os.makedirs('data',
exist_ok=True) # create folder (no error if exists) print(os.listdir('.')) # list files in
current directory print(os.path.exists('notes.txt')) # True or False print(os.path.join('data',
'file.txt')) # safe path: data/file.txt # Get file info size = os.path.getsize('notes.txt') #
size in bytes name = os.path.basename('/home/user/data/file.txt') # 'file.txt' folder =
os.path.dirname('/home/user/data/file.txt') # '/home/user/data'
```

■ Run each snippet. After each write, open the file in VS Code to see the result.

Lab — Day 1

Create **day1_fileio.py**. Build a simple note-taking program:

- Create a folder called **data/** using `os.makedirs()`
- Write a function **save_note(filename, text)** that appends a line with a timestamp to a file in `data/`
- Import `datetime`: **from datetime import datetime** — use `datetime.now().strftime('%Y-%m-%d %H:%M')` for the timestamp
- Write a function **read_notes(filename)** that prints all lines from the file
- Use a while loop to keep asking the user: "1=Add note, 2=Read notes, 3=Quit"
- Handle the case where the notes file does not exist yet (use `os.path.exists()`)

■ Commit: 'feat: day 1 file I/O lab — note-taking app'

AWS Setup (Do This Today — 30 min)

- Create a free AWS account at **aws.amazon.com** if you don't have one
- Enable MFA on the root account immediately (Security Credentials → MFA)
- Create an IAM user called **miryam-admin** with AdministratorAccess policy
- Download the IAM user's access keys — store them safely, never in GitHub
- Install AWS CLI: **brew install awscli** (Mac) then run: **aws configure**
- Verify: **aws sts get-caller-identity** — should print your account ID
- Sign up for Stephane Maarek's AWS Cloud Practitioner course on Udemy (get it on sale)

DAY 2 · Tuesday, August 4

Exception Handling — try / except / finally

Real programs fail. Learn to handle errors gracefully instead of crashing.

Lesson: Why Exceptions Matter

An exception is Python's way of saying "something went wrong." Without handling them, your program crashes with a traceback. In production code — and in your projects — you must always anticipate what can go wrong.

```
# Without exception handling – program crashes on bad input age = int(input('Enter your age: ')) # crashes if user types 'abc' # With exception handling – program stays alive try: age = int(input('Enter your age: ')) print(f'You are {age} years old.') except ValueError: print('Please enter a valid number.') # Multiple except blocks – catch different error types try: number = int(input('Enter a number: ')) result = 10 / number print(result) except ValueError: print('That is not a number.') except ZeroDivisionError: print('Cannot divide by zero.') except Exception as e: print(f'Unexpected error: {e}') # catch-all # finally – always runs, even if exception occurred try: f = open('data.txt', 'r') content = f.read() except FileNotFoundError: print('File not found.') finally: print('This always runs – good for cleanup.')
```

Lesson: else Clause + Raising Exceptions

```
# else runs only if NO exception occurred try: result = int('42') except ValueError: print('Bad value') else: print(f'Success: {result}') # This runs – no exception happened # Raise your own exceptions def get_age(age): if age < 0 or age > 150: raise ValueError(f'Invalid age: {age}') return age try: get_age(-5) except ValueError as e: print(e) # Invalid age: -5 # Custom exception classes class InvalidInputError(Exception): pass def validate_password(pw): if len(pw) < 8: raise InvalidInputError('Password must be at least 8 characters.')
```

■ *Exception handling is not optional — every project from now on must use it on user input and file operations.*

Lab — Day 2

Create **day2_exceptions.py**. Write THREE programs:

Program 1 — Safe Calculator:

- Ask user for two numbers and an operator (+, -, *, /)
- Wrap in try/except — handle ValueError (non-number input) and ZeroDivisionError
- Keep looping until the user types "quit"

Program 2 — Safe File Reader:

- Ask user for a filename
- Try to open and print it
- Handle FileNotFoundError with a friendly message
- Handle PermissionError separately

Program 3 — Validated Input Function:

- Write a function **get_positive_int(prompt)** that keeps asking until the user enters a positive integer

- Use a while True loop + try/except inside

■ Commit: 'feat: day 2 exception handling lab'

AWS Study — Cloud Concepts (Watch 1 hr today)

- Watch: "What is Cloud Computing?" section of your Udemy course
- Understand: **IaaS** (you manage OS+), **PaaS** (you manage app only), **SaaS** (you use it)
- Memorize the 6 benefits of cloud: trade CapEx for OpEx, massive economies of scale, stop guessing capacity, increase speed and agility, stop spending on data centers, go global in minutes
- Understand the **Shared Responsibility Model**: AWS owns hardware/datacenters; YOU own your data/OS/app

DAY 3 · Wednesday, August 5

JSON — The Language of APIs

JSON is how the internet sends data. You will use it every single day as a cloud engineer.

Lesson: What is JSON?

JSON (JavaScript Object Notation) is a text format for storing and exchanging data. It looks like a Python dictionary. AWS APIs, REST APIs, config files — everything uses JSON.

```
import json # Python dict <-> JSON string
engineer = { "name": "Miryam", "role": "Cloud Engineer", "skills": ["Python", "AWS"], "certified": False, "salary": None } # dict -> JSON string (serialise)
json_string = json.dumps(engineer)
print(json_string) # '{"name": "Miryam", "role": "Cloud Engineer", ...}' # Pretty-print JSON (use indent=)
print(json.dumps(engineer, indent=2)) # JSON string -> dict (deserialise)
data = json.loads(json_string)
print(data["name"]) # Miryam
print(type(data)) # <class 'dict'> # JSON type mapping: # Python dict -> JSON object {} # Python list -> JSON array [] # Python str -> JSON string "" # Python int/float -> JSON number # Python True/False -> true/false # Python None -> null
```

Lesson: Reading and Writing JSON Files

```
import json # Write dict to a JSON file
data = {"users": ["Alice", "Bob"], "count": 2}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2) # json.dump (not .dumps) for files
# Read JSON from a file
with open("data.json", "r") as f:
    loaded = json.load(f) # json.load (not .loads) for files
print(loaded["count"]) # 2
# Pattern: load -> modify -> save (used in every project)
def load_data(filepath):
    if not os.path.exists(filepath):
        return []
    with open(filepath, "r") as f:
        return json.load(f)
def save_data(filepath, data):
    with open(filepath, "w") as f:
        json.dump(data, f, indent=2)
```

■ The load/modify/save pattern is the foundation of your Expense Tracker and Contact Book projects.

Lab — Day 3

Create **day3_json.py**. Build a JSON data store:

- Create a file **students.json** with a list of 3 student dictionaries (name, grade, gpa)
- Write a function **add_student(name, grade, gpa)** that loads the file, appends a new student, and saves it back
- Write a function **find_student(name)** that searches by name and returns the student dict or None
- Write a function **print_honor_roll()** that prints all students with gpa >= 3.5
- Wrap all file operations in try/except for JSONDecodeError and FileNotFoundError

■ Commit: 'feat: day 3 JSON lab — student data store'

AWS Study — IAM Fundamentals (30 min)

- **IAM** = Identity and Access Management — controls WHO can do WHAT on AWS
- Key components: **Users** (people), **Groups** (collections of users), **Roles** (for services/apps), **Policies** (JSON documents defining permissions)
- **Least privilege**: always grant minimum permissions required. Never use root for daily work.

- Practice: In AWS Console → IAM → create a group called "developers" → attach ReadOnlyAccess policy

DAY 4 · Thursday, August 6

List & Dictionary Comprehensions

Write powerful one-liners. This is what separates beginner Python from professional Python.

Lesson: List Comprehensions

A list comprehension creates a new list by applying an expression to each item in an iterable. Cleaner and faster than a for loop + append.

```
# Standard for loop squares = [] for n in range(1, 6): squares.append(n ** 2) # Same thing as
list comprehension squares = [n ** 2 for n in range(1, 6)] # [1, 4, 9, 16, 25] # With a
condition (filter) evens = [n for n in range(20) if n % 2 == 0] # [0, 2, 4, 6, 8, 10, 12, 14,
16, 18] # String operations names = ["alice", "bob", "charlie"] upper = [name.upper() for name
in names] # ['ALICE', 'BOB', 'CHARLIE'] lengths = [len(name) for name in names] # [5, 3, 7] #
Nested comprehension (use sparingly – readability matters) matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [n for row in matrix for n in row] # [1, 2, 3, 4, 5, 6, 7, 8, 9] # Conditional expression
inside comprehension labels = ["pass" if score >= 60 else "fail" for score in [85, 45, 72, 55]]
# ['pass', 'fail', 'pass', 'fail']
```

Lesson: Dictionary Comprehensions

```
# Dict comprehension: {key: value for item in iterable} names = ["Alice", "Bob", "Charlie"]
name_lengths = {name: len(name) for name in names} # {'Alice': 5, 'Bob': 3, 'Charlie': 7} #
Filter a dictionary (only keep entries where value > 4) filtered = {k: v for k, v in
name_lengths.items() if v > 4} # {'Alice': 5, 'Charlie': 7} # Invert a dictionary (swap keys
and values) original = {"a": 1, "b": 2, "c": 3} inverted = {v: k for k, v in original.items()}
# {1: 'a', 2: 'b', 3: 'c'} # Build a lookup dict from a list of dicts users = [{"id": 1,
"name": "Alice"}, {"id": 2, "name": "Bob"}] lookup = {u["id"]: u["name"] for u in users} # {1:
'Alice', 2: 'Bob'}
```

Lab — Day 4

Create **day4_comprehensions.py** and solve 5 problems using comprehensions — no for loops with append():

- Given a list of temperatures in Celsius, create a list of Fahrenheit values
- Given a list of words, create a list of words longer than 4 characters
- Given a list of numbers, create a dict mapping each number to its cube
- Given a list of student dicts (name, grade), create a list of names where grade >= 70
- Given a sentence, create a dict of {word: count} for each unique word

■ Commit: 'feat: day 4 comprehensions lab'

AWS Study — EC2 Basics (30 min)

- **EC2** = Elastic Compute Cloud — virtual servers in the cloud
- Key concepts: **Instance types** (t3.micro = small/cheap), **AMIs** (OS blueprints), **Security Groups** (virtual firewall — controls inbound/outbound traffic)
- **Key pairs**: SSH key used to log into Linux instances — never lose the .pem file

- **Pricing models:** On-Demand (pay by hour), Reserved (1-3yr commitment = cheaper), Spot (cheapest, can be interrupted)

DAY 5 · Friday, August 7

Python Modules — os, datetime, sys, math

Stop reinventing the wheel. Python's standard library does the heavy lifting.

Lesson: The os Module

```
import os # Environment variables (used in cloud config) path = os.environ.get('PATH', 'not set') secret = os.environ.get('API_KEY', 'missing') # safe default # Setting environment variables (for testing only – use .env in production) os.environ['MY_VAR'] = 'hello' # File system cwd = os.getcwd() home = os.path.expanduser('~') os.rename('old.txt', 'new.txt') os.remove('file.txt') os.makedirs('a/b/c', exist_ok=True) # creates nested dirs # Walk a directory tree for root, dirs, files in os.walk('my_folder'): for file in files: full_path = os.path.join(root, file) print(full_path)
```

Lesson: The datetime Module

```
from datetime import datetime, date, timedelta # Current date and time now = datetime.now() today = date.today() print(now) # 2026-08-07 09:30:00.123456 print(today) # 2026-08-07 # Formatting – strftime (string FROM time) stamp = now.strftime('%Y-%m-%d %H:%M:%S') # '2026-08-07 09:30:00' stamp2 = now.strftime('%B %d, %Y') # 'August 07, 2026' # Parsing – strptime (string TO parsed time) dt = datetime.strptime('2026-08-31', '%Y-%m-%d') # Arithmetic with timedelta tomorrow = today + timedelta(days=1) last_week = today - timedelta(weeks=1) exam_date = date(2026, 8, 31) days_left = (exam_date - today).days print(f'Days until exam: {days_left}')
```

Lab — Day 5

Create **day5_modules.py**. Build a file organiser:

- Use **os.listdir()** to list all files in the current directory
- Categorize files by extension: .py → Python, .txt → Text, .json → Data, other → Other
- Print a summary: how many files in each category
- For each file, print: filename, size in KB (os.path.getsize / 1024), and last modified time (use os.path.getmtime + datetime.fromtimestamp())
- Use **datetime.now()** to print: "Report generated on [date]"

■ Commit: 'feat: day 5 modules lab — file organizer'

AWS Study — S3 Fundamentals (30 min)

- **S3** = Simple Storage Service — object storage (not a file system)
- **Bucket**: globally unique container for objects. **Object**: any file + metadata.
- **Storage classes**: Standard (frequent access), Standard-IA (infrequent), Glacier (archive)
- **Versioning**: keeps all versions of an object — protects against accidental deletion
- Practice: Create an S3 bucket in the console (use your name + date as bucket name), upload a text file, view the URL

DAY 6 · Saturday, August 8

START Expense Tracker + AWS APIs with requests

Your third project begins today. Connect Python to the internet.

Lesson: The requests Library

The requests library lets Python talk to REST APIs. It is not in the standard library — install it first.

```
# Install: pip3 install requests import requests # Basic GET request response =
requests.get('https://api.github.com/users/octocat') # Check status print(response.status_code)
# 200 = success, 404 = not found, etc. response.raise_for_status() # raises exception if status
>= 400 # Parse JSON response data = response.json() # returns a Python dict print(data['name'])
print(data['public_repos']) # GET with query parameters params = {'q': 'python', 'sort':
'stars'} r = requests.get('https://api.github.com/search/repositories', params=params) results
= r.json() print(results['total_count']) # Always wrap API calls in try/except try: r =
requests.get('https://api.example.com/data', timeout=10) r.raise_for_status() data = r.json()
except requests.exceptions.ConnectionError: print('No internet connection.') except
requests.exceptions.Timeout: print('Request timed out.') except requests.exceptions.HTTPError
as e: print(f'HTTP error: {e}')
```

START Project 3 — Expense Tracker (Due Monday Aug 10)

■ **DEADLINE: Expense Tracker — COMPLETE by Monday August 10**

Create `expense_tracker.py`. Requirements:

- **Data structure:** each expense = {'date': '2026-08-08', 'category': 'Food', 'amount': 12.50, 'description': 'Lunch'}
- Store all expenses in a JSON file: `expenses.json`
- Functions required: `add_expense(category, amount, description)`, `view_all()`, `view_by_category(cat)`, `total_spent()`, `monthly_summary()`
- Menu: 1=Add expense, 2=View all, 3=View by category, 4=Monthly summary, 5=Total spent, 6=Quit
- Input validation: reject negative amounts, reject empty category
- Use `datetime.today().strftime('%Y-%m-%d')` to auto-stamp each expense

AWS Study — VPC Basics (30 min)

- **VPC** = Virtual Private Cloud — your isolated network in AWS
- **Subnets:** public (internet-accessible) vs private (internal only)
- **Internet Gateway:** allows public subnet resources to reach the internet
- **NAT Gateway:** allows private subnet resources to initiate outbound internet connections (but blocks inbound)
- **Route Tables:** rules that say "send traffic for X to Y"

SUNDAY · August 9 — REST DAY

Light review only · Finalize Expense Tracker · AWS flashcard review

Rest Day tasks (light, 45 min max):

- Finish any incomplete code in `expense_tracker.py` — it is due tomorrow
- Review IAM, EC2, S3, VPC notes — write 3 key facts from each on paper
- Do 10 AWS Cloud Practitioner practice questions at **examtopics.com** or Udemy quizzes
- Optional: watch the VPC section of your Udemy course (30 min)

DAY 7 · Monday, August 10

Expense Tracker DUE + OOP Part 1 — Classes

Project 3 is due today. Then level up your Python with object-oriented programming.

Project 3 — Expense Tracker (DUE TODAY)

■ **DEADLINE: Expense Tracker: August 10**

Your `expense_tracker.py` must include ALL of the following before you commit:

- All 6 menu options working correctly
- JSON persistence — expenses survive between program runs
- Input validation with try/except on all user inputs
- `monthly_summary()` groups expenses by month and shows subtotals per category
- A professional **README.md** explaining the project and how to run it

```
git add expense_tracker.py README.md git commit -m 'feat: project 3 expense tracker complete'
git push
```

Lesson: Classes and OOP

A class is a blueprint for creating objects. Objects bundle data (attributes) and behaviour (methods) together. OOP makes large programs easier to organise and reason about.

```
# Define a class class CloudServer: # __init__ runs when you create an instance (constructor)
def __init__(self, name, instance_type, region): self.name = name # instance attribute
self.instance_type = instance_type self.region = region self.running = False # default value #
Method (function inside a class – always takes self as first param) def start(self):
self.running = True print(f'{self.name} is now running in {self.region}') def stop(self):
self.running = False print(f'{self.name} stopped.') def status(self): state = 'RUNNING' if
self.running else 'STOPPED' return f'{self.name} ({self.instance_type}) – {state}' # __str__
controls what print(object) shows def __str__(self): return self.status() # Create instances
web = CloudServer('web-01', 't3.micro', 'us-east-1') db = CloudServer('db-01', 'm5.large',
'us-east-1') web.start() print(web) # web-01 (t3.micro) – RUNNING print(db.running) # False
```

Lab — Day 7

Create `day7_oop.py`. Write TWO classes:

Class 1 — BankAccount: attributes: owner, balance. Methods: `deposit(amount)`, `withdraw(amount)` (reject if insufficient funds), `get_balance()`, `__str__`

Class 2 — Library: attributes: name, books=[] (list). Methods: `add_book(title, author)`, `remove_book(title)`, `find_book(title)`, `list_all()`, `__str__`

■ Commit: 'feat: day 7 OOP classes lab'

AWS Study — RDS and Lambda (30 min)

- **RDS** = Relational Database Service — managed MySQL, PostgreSQL, MariaDB, Oracle, SQL Server
- **Multi-AZ**: AWS keeps a synchronous standby in another AZ. Automatic failover. Use for high availability.
- **Read Replica**: async copy for scaling reads. Does NOT provide failover.
- **Lambda**: serverless compute. Write a function, AWS runs it. Pay only for invocations.
- Lambda triggers: S3 events, API Gateway, CloudWatch Events, SQS, DynamoDB Streams

DAY 8 · Tuesday, August 11

OOP Part 2 — Inheritance + Special Methods

Build on top of existing classes. This is how real-world Python frameworks work.

Lesson: Inheritance

Inheritance lets you create a new class that gets all the attributes and methods of an existing class, then add or override as needed. Write shared code once.

```
# Base class (parent) class Animal: def __init__(self, name, species): self.name = name
self.species = species def speak(self): return f'{self.name} makes a sound.' def __str__(self):
return f'{self.name} ({self.species})' # Derived class (child) – inherits from Animal class
Dog(Animal): def __init__(self, name, breed): super().__init__(name, 'Canis lupus') # call
parent __init__ self.breed = breed def speak(self): # override parent method return
f'{self.name} barks!' def fetch(self): # new method, only on Dog return f'{self.name} fetches
the ball.' class Cat(Animal): def speak(self): return f'{self.name} meows.' # Polymorphism –
same method name, different behaviour animals = [Dog('Rex', 'Labrador'), Cat('Whiskers',
'tabby')] for animal in animals: print(animal.speak()) # each calls its own speak() # Cloud
engineering example: base resource class class AWSResource: def __init__(self, resource_id,
region): self.resource_id = resource_id self.region = region def get_arn(self): raise
NotImplementedError('Subclass must implement get_arn()') class S3Bucket(AWSResource): def
__init__(self, name, region): super().__init__(name, region) self.name = name def
get_arn(self): return f'arn:aws:s3:::{self.name}'
```

Lab — Day 8

Create **day8_inheritance.py**. Build a class hierarchy:

- Base class **Employee(name, department, salary)** with methods: `get_info()`, `apply_raise(percent)`
- Subclass **Engineer(name, salary, specialization)** — inherits `Employee`, sets `department="Engineering"`, adds method `get_tech_stack()`
- Subclass **Manager(name, salary, team_size)** — inherits `Employee`, adds `manage_team()` method
- Create 3 instances, store in a list, loop and call `get_info()` on each
- Use `isinstance(obj, Manager)` to print a special message for managers only

■ Commit: 'feat: day 8 inheritance lab'

AWS Study — CloudWatch + Billing (30 min)

- **CloudWatch**: monitoring and observability. Collects metrics (CPU, memory, requests), logs, and events.
- **Alarms**: trigger when a metric crosses a threshold (e.g. CPU > 80% for 5 minutes)
- **Free Tier**: AWS gives new accounts 12 months of free usage on many services (t2.micro EC2, 5GB S3, etc.)
- **Cost Explorer**: analyse your spending over time
- **AWS Budgets**: set a dollar limit and get email alerts — DO THIS NOW to avoid surprise bills

■ Set up a \$5 billing alert in AWS Budgets right now — takes 2 minutes

DAY 9 · Wednesday, August 12

Calling REST APIs — requests Deep Dive

Every cloud tool, every modern app, talks to APIs. Master this skill today.

Lesson: API Headers, Auth, and POST Requests

```
import requests # GET with headers (for APIs that require API keys) headers = {
    'Authorization': 'Bearer YOUR_API_KEY', 'Content-Type': 'application/json' } r =
requests.get('https://api.openweathermap.org/data/2.5/weather', params={'q': 'London', 'appid':
    'YOUR_KEY'}, timeout=10) # POST request (send data to a server) payload = {'title': 'New Post',
    'body': 'Content here', 'userId': 1} r =
requests.post('https://jsonplaceholder.typicode.com/posts', json=payload, # automatically sets
    Content-Type: application/json timeout=10) print(r.status_code) # 201 = Created print(r.json())
# response data # Response object attributes print(r.status_code) # 200, 201, 404, etc.
print(r.headers) # response headers (dict) print(r.text) # raw response text print(r.json()) #
    parsed JSON (dict/list) print(r.url) # the full URL that was requested # Free practice APIs (no
    key needed): # https://jsonplaceholder.typicode.com - fake REST API # https://api.github.com -
    GitHub # https://wttr.in/London?format=json - weather (JSON) # https://catfact.ninja/fact -
    random cat facts
```

Lesson: Environment Variables for API Keys

Never hardcode API keys in your code. Use environment variables:

```
import os from dotenv import load_dotenv # pip3 install python-dotenv load_dotenv() # reads
    .env file api_key = os.environ.get('OPENWEATHER_API_KEY') if not api_key: raise
    EnvironmentError('OPENWEATHER_API_KEY not set in environment') # .env file (NEVER commit this
    to GitHub!) # OPENWEATHER_API_KEY=your_key_here # DATABASE_URL=postgresql://user:pass@host/db #
    Add .env to your .gitignore: # .env # *.env
```

Lab — Day 9

Create **day9_apis.py**. Call two public APIs:

Part 1 — GitHub API:

- Fetch your GitHub profile: GET https://api.github.com/users/YOUR_USERNAME
- Print: name, public repos count, followers, and when the account was created
- Fetch your repos: GET https://api.github.com/users/YOUR_USERNAME/repos
- Print a table: repo name | language | stars | last updated

Part 2 — Random Facts API:

- GET <https://catfact.ninja/fact> — print 5 random facts using a loop
- Wrap in try/except for connection errors and timeouts

■ Commit: 'feat: day 9 APIs lab — GitHub and random facts'

AWS Study — Global Infrastructure (20 min)

- **Regions:** geographic locations (us-east-1, eu-west-1, ap-southeast-1). Choose based on latency + compliance.
- **Availability Zones (AZ):** 2-6 isolated data centers per region. Deploy across AZs for high availability.
- **Edge Locations:** 200+ points of presence for CloudFront CDN caching
- Always deploy across multiple AZs. Never rely on a single AZ.

DAY 10 · Thursday, August 13

Python CLI Patterns + Contact Book Start

Professional Python programs have proper command-line interfaces. Build one today.

Lesson: Building Clean CLI Programs

```
import sys import argparse # argparse – Python's built-in CLI argument parser parser =
argparse.ArgumentParser(description='Expense Tracker CLI') parser.add_argument('action',
choices=['add', 'list', 'total'], help='Action to perform') parser.add_argument('--amount',
type=float, help='Expense amount') parser.add_argument('--category', default='General',
help='Expense category') parser.add_argument('--desc', default='', help='Description') args =
parser.parse_args() # Run with: python3 app.py add --amount 12.50 --category Food --desc Lunch
if args.action == 'add': if args.amount is None: print('Error: --amount required for add')
sys.exit(1) # add the expense... # sys.argv – raw command line arguments print(sys.argv) #
['script.py', 'add', '--amount', '12.50'] print(sys.argv[0]) # script name print(sys.argv[1:])
# all arguments after script name # Clean menu pattern (reusable function) def
show_menu(options: dict): print() for key, label in options.items(): print(f' {key}. {label}')
return input('Choose: ').strip()
```

START Project 4 — Contact Book (Due Monday Aug 17)

■ **DEADLINE: Contact Book — COMPLETE by Monday August 17**

Create **contact_book.py**. This is a full CRUD app stored in JSON:

- **Data model:** each contact = {"id": 1, "name": "Alice", "phone": "555-1234", "email": "alice@email.com", "notes": ""}
- **CRUD operations:** Create (add), Read (view/search), Update (edit), Delete (remove)
- **Functions:** add_contact(), search_by_name(query), update_contact(id, field, value), delete_contact(id), list_all(), export_to_txt()
- IDs: auto-increment (find max id in existing data + 1)
- search_by_name should be case-insensitive and support partial matches
- Use a JSON file (contacts.json) for storage — load on start, save on every change
- Write a professional README.md

AWS Study — Well-Architected Framework (30 min)

The 6 pillars of the AWS Well-Architected Framework — memorize these for the exam:

- **Operational Excellence:** run and monitor systems, continually improve processes
- **Security:** protect data and systems, use IAM, encrypt data
- **Reliability:** recover from failures, scale horizontally, test recovery procedures
- **Performance Efficiency:** use resources efficiently, choose right instance types
- **Cost Optimization:** avoid unnecessary costs, use right pricing models
- **Sustainability:** minimize environmental impact, maximize utilization

DAY 11 · Friday, August 14

OOP Applied — Classes in Real Projects

Refactor your Expense Tracker to use OOP. This is how production code is written.

Lesson: Class Methods + Static Methods

```
class Expense: # Class variable – shared across all instances category_list = ['Food',
    'Transport', 'Housing', 'Entertainment', 'Other'] def __init__(self, amount, category,
    description, date=None): from datetime import date as d self.amount = float(amount)
    self.category = category self.description = description self.date = date or
    d.today().isoformat() self.id = id(self) # simple unique id # Class method – works on the
    class, not an instance @classmethod def from_dict(cls, data: dict): return cls(data['amount'],
    data['category'], data['description'], data['date']) # Static method – utility function, no
    self or cls @staticmethod def validate_amount(amount) -> bool: try: return float(amount) > 0
    except (TypeError, ValueError): return False def to_dict(self) -> dict: return {'amount':
    self.amount, 'category': self.category, 'description': self.description, 'date': self.date} def
    __str__(self): return f'[{self.date}] {self.category}: ${self.amount:.2f} – {self.description}'
    # Usage e = Expense(12.50, 'Food', 'Lunch') print(e) # [2026-08-14] Food: $12.50 – Lunch e2 =
    Expense.from_dict({'amount': 5.0, 'category': 'Transport', 'description': 'Bus', 'date':
    '2026-08-14'}) print(Expense.validate_amount(-1)) # False
```

Lab — Day 11

Create `day11_oop_refactor.py`:

- Refactor your `expense_tracker` into two classes: **Expense** (single expense) and **ExpenseTracker** (manages the list + file)
- `ExpenseTracker` methods: `add(expense)`, `remove(id)`, `get_by_category(cat)`, `total()`, `monthly_summary()`, `save()`, `load()`
- `ExpenseTracker.__init__` should call `self.load()` automatically
- All file I/O only inside `save()` and `load()` — no file operations in other methods

■ Commit: 'refactor: expense tracker with OOP classes'

AWS Study — Practice Questions (30 min)

Go to examtopics.com/exams/amazon/clf-c02/ or use Udemy practice tests. Complete 20 questions. For every wrong answer, write down the correct answer and WHY it is correct.

DAY 12 · Saturday, August 15

Contact Book Work + AWS Security Deep Dive

Polish your Contact Book. Understand AWS security — it's 25% of the CLF exam.

Contact Book — Complete These Features Today

- All CRUD operations working and tested manually
- `search_by_name()` returns partial, case-insensitive matches
- `export_to_txt()` writes a clean, formatted text report of all contacts
- Handle duplicate names gracefully (allow, but warn the user)
- Validate email format (basic check — must contain @ and .)

■ *Target: Contact Book should be 90% done by end of today.*

Lesson: Python Regular Expressions (Bonus)

```
import re # Validate email def is_valid_email(email: str) -> bool: pattern =
r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$' return bool(re.match(pattern, email))
print(is_valid_email('alice@gmail.com')) # True print(is_valid_email('notanemail')) # False #
Validate phone (US format: 555-1234 or 555-123-4567) def is_valid_phone(phone: str) -> bool:
pattern = r'\d{3}-\d{3,4}-?\d{0,4}' return bool(re.match(pattern, phone)) # re.search — find
pattern anywhere in string text = 'Error: connection refused on port 5432' match =
re.search(r'port (\d+)', text) if match: print(match.group(1)) # '5432' # re.findall — return
all matches as a list emails = re.findall(r'[\w.-]+@[ \w.- ]+', 'Contact alice@x.com or
bob@y.com') # ['alice@x.com', 'bob@y.com']
```

AWS Study — IAM Security Deep Dive (30 min)

- **IAM Policies:** JSON documents. Effect (Allow/Deny), Action (s3:GetObject), Resource (ARN), Condition
- **Permission boundaries:** set the maximum permissions a user/role can have — even if a policy tries to grant more
- **Service Control Policies (SCPs):** org-level policies that apply to all accounts in AWS Organizations
- **MFA:** always enable on root and all IAM users
- **Access Analyzer:** identifies resources shared with external entities
- **CloudTrail:** logs every API call ever made to your account — audit trail

SUNDAY · August 16 — REST DAY

Light review · Finish Contact Book · 20 AWS practice questions

Rest Day tasks (45 min max):

- Finalize `contact_book.py` — it is due tomorrow morning
- Review your notes: file I/O, exceptions, JSON, comprehensions, OOP, requests
- Do 20 more CLF-C02 practice questions — aim for 75%+ accuracy
- Optional: rewatch any AWS lecture section you found confusing

DAY 13 · Monday, August 17

Contact Book DUE + Weather API App Start

Project 4 ships today. Then build your most impressive project yet.

Project 4 — Contact Book (DUE TODAY)

■ **DEADLINE: Contact Book: August 17**

Your `contact_book.py` must have ALL of the following before committing:

- All CRUD operations: add, view, search, update, delete
- Persistent JSON storage — data survives between runs
- Case-insensitive partial search by name
- Input validation: email format check, no empty required fields
- `export_to_txt()` generates a readable contact list file
- Professional README.md with features list and usage instructions

```
git add contact_book.py README.md contacts.json git commit -m 'feat: project 4 contact book complete' git push
```

START Project 5 — Weather API App (Due Saturday Aug 24... updated: Aug 22)

■ **DEADLINE: Weather API App — COMPLETE by Saturday August 22**

Create `weather_app.py`. Requirements:

- Sign up for a free API key at **openweathermap.org** (free tier is enough)
- Store your API key in a `.env` file + use `python-dotenv` to load it — NEVER commit the key
- Function `get_weather(city)`: calls OpenWeatherMap current weather endpoint, returns a parsed dict
- Function `get_forecast(city, days=5)`: calls the 5-day forecast endpoint
- Display: city, temperature (C and F), feels like, humidity, wind speed, weather description
- Error handling: city not found (404), no internet (ConnectionError), missing API key
- Bonus: save last 10 searches to a JSON file so user can re-query recent cities

```
# OpenWeatherMap endpoint: #
https://api.openweathermap.org/data/2.5/weather?q={city}&appid={key}&units=metric # Useful
fields: data['main']['temp'], data['weather'][0]['description'], # data['main']['humidity'],
data['wind']['speed']
```

AWS Study — EC2 Deep Dive (30 min)

- **Security Groups**: stateful firewall. Inbound rules: what can reach your instance. Outbound: what it can reach.
- **User Data**: bash script that runs on instance launch — used to install software automatically
- **Instance metadata**: `http://169.254.169.254/latest/meta-data/` — get instance info from inside the instance
- **Elastic IP**: static public IP address that stays the same even when instance restarts

DAY 14 · Tuesday, August 18

Weather API App — Build It Out

APIs + JSON + error handling + environment variables — all in one project.

Lesson: Working with the OpenWeatherMap API

```
import requests, os from dotenv import load_dotenv load_dotenv() API_KEY =
os.environ.get('OPENWEATHER_API_KEY') BASE_URL = 'https://api.openweathermap.org/data/2.5' def
get_weather(city: str) -> dict: url = f'{BASE_URL}/weather' params = {'q': city, 'appid':
API_KEY, 'units': 'metric'} try: r = requests.get(url, params=params, timeout=10)
r.raise_for_status() data = r.json() return { 'city': data['name'], 'country':
data['sys']['country'], 'temp_c': data['main']['temp'], 'temp_f': round(data['main']['temp'] *
9/5 + 32, 1), 'feels_like': data['main']['feels_like'], 'humidity': data['main']['humidity'],
'wind_speed': data['wind']['speed'], 'description': data['weather'][0]['description'].title() }
except requests.exceptions.HTTPError as e: if r.status_code == 404: raise ValueError(f'City not
found: {city}') raise def display_weather(weather: dict): print(f'\n{weather["city"]},
{weather["country"]}') print(f' Temperature: {weather["temp_c"]}°C / {weather["temp_f"]}°F')
print(f' Feels like: {weather["feels_like"]}°C') print(f' Humidity: {weather["humidity"]}%)')
print(f' Wind: {weather["wind_speed"]} m/s') print(f' Conditions: {weather["description"]}')
```

■ Run this today. Get real weather data printing to your terminal.

Lab — Day 14

Complete at least these parts of weather_app.py today:

- get_weather(city) working and returning clean data
- display_weather() printing a nicely formatted output
- Full error handling: city not found, no internet, missing API key
- Search history: save each city searched to weather_history.json

■ Commit: 'feat: weather app — current weather working'

AWS Study — S3 Advanced (30 min)

- **Bucket policies:** JSON resource-based policies attached to a bucket — control who can access it
- **Static website hosting:** Enable on an S3 bucket, upload index.html, make public
- **S3 versioning:** keeps all versions — costs more storage but protects against deletion/overwrites
- **Lifecycle rules:** automatically move objects to cheaper storage after N days, or delete them
- **Pre-signed URLs:** temporary access URLs — share a private object for 1 hour without making it public

DAY 15 · Wednesday, August 19

5-Day Forecast + AWS Practice Exam #1

Finish the weather app's forecast feature. Then take your first full practice exam.

Lesson: Parsing Forecast Data

```
from collections import defaultdict from datetime import datetime def get_forecast(city: str,
days: int = 5) -> list: url = f'{BASE_URL}/forecast' params = {'q': city, 'appid': API_KEY,
'units': 'metric', 'cnt': days * 8} r = requests.get(url, params=params, timeout=10)
r.raise_for_status() data = r.json() # Group by day daily = defaultdict(list) for item in
data['list']: day = item['dt_txt'].split(' ')[0] # '2026-08-20' daily[day].append(item)
forecast = [] for day, readings in list(daily.items())[:days]: temps = [r['main']['temp'] for r
in readings] descs = [r['weather'][0]['description'] for r in readings] forecast.append({
'date': day, 'high': round(max(temps), 1), 'low': round(min(temps), 1), 'summary':
max(set(descs), key=descs.count) # most common }) return forecast def
display_forecast(forecast: list): print('\n5-Day Forecast:') print('-' * 40) for day in
forecast: dt = datetime.strptime(day['date'], '%Y-%m-%d').strftime('%a %b %d') print(f'{dt}:
{day["low"]}°C - {day["high"]}°C {day["summary"].title()}')
```

AWS Practice Exam #1 (Do This Today)

■ Take a full 65-question CLF-C02 practice exam today. Target score: 70%+

- Use Udemy practice tests (Stephane Maarek or Jon Bonso) — 65 questions, timed to 90 min
- When you finish, review EVERY wrong answer — understand why the correct answer is right
- Write down the top 5 topics you got wrong — those are your weak spots to focus on
- Score < 70%: revisit those AWS topic notes before moving on

■ Commit weather app progress: 'feat: weather app — 5-day forecast added'

DAY 16 · Thursday, August 20

Weather App Polish + AWS Weak Spot Review

Fix what the practice exam exposed. Finish the weather app.

Weather App — Final Requirements Checklist

- `get_weather(city)` → returns structured dict
- `get_forecast(city, days=5)` → returns list of daily forecasts
- `display_weather()` and `display_forecast()` print clean formatted output
- Full error handling on all API calls
- Search history in `weather_history.json`
- Units: shows both Celsius and Fahrenheit
- README.md with setup instructions (how to get API key, how to set .env file)

■ *Final commit for weather app: 'feat: project 5 weather API app complete'*

Lab — Day 16: AWS Weak Spot Drills

Based on your practice exam results, spend 90 min deep-diving the topics you got wrong. Use this structure:

- Read the AWS documentation page for the service you missed (just the overview + FAQs)
- Find 10 practice questions specifically about that topic
- Write a 3-sentence explanation of the concept in your own words — if you can't explain it, you don't know it

AWS Study — Route 53 + CloudFront (30 min)

- **Route 53:** AWS DNS service. Routing policies: Simple (one target), Weighted (split traffic %), Latency (closest region), Failover (primary/backup), Geolocation (by country)
- **CloudFront:** AWS CDN — caches content at edge locations worldwide. Reduces latency, absorbs DDoS.
- CloudFront + S3 = deliver static websites globally with HTTPS and low latency

DAY 17 · Friday, August 21

S3 Static Website — Build Your First Cloud Deployment

Stop running code locally. Deploy it to the cloud. This is what cloud engineering looks like.

Project 6 — S3 Static Website (Due Friday Aug 28)

■ **DEADLINE: S3 Static Website — COMPLETE by Friday August 28**

Host a professional HTML/CSS website on AWS S3 with static website hosting:

- Create a simple but polished HTML portfolio page: your name, bio, skills list, GitHub link, projects list (BMI Calc, Password Gen, Expense Tracker, Contact Book, Weather App)
- CSS: make it look professional — use a Google Font, add a hero section, use a colour scheme
- Create S3 bucket: **miryam-portfolio-2026** (must be globally unique)
- Enable Static Website Hosting in bucket properties (set index.html as index document)
- Set bucket policy to allow public read access (s3:GetObject for all principals)
- Upload your HTML and CSS files
- Access your site at the S3 website endpoint URL and verify it works
- Add a README.md with the live URL and deployment instructions

S3 Bucket Policy (Copy This)

```
{ "Version": "2012-10-17", "Statement": [ { "Sid": "PublicReadGetObject", "Effect": "Allow",  
  "Principal": "*", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*" } ] }
```

AWS Study — Lambda Deep Dive (30 min)

- **Lambda execution model:** cold start vs warm start, concurrency, reserved concurrency
- **Lambda layers:** shared dependencies (like your Python libraries) across functions
- **Environment variables:** store config/secrets — encrypt with KMS
- **Lambda@Edge:** run Lambda functions at CloudFront edge locations
- **Pricing:** 1 million free requests/month + 400,000 GB-seconds compute — very cheap for small workloads

DAY 18 · Saturday, August 22

Weather App DUE + Practice Exam #2

Weather App ships today. Take your second full practice exam. You should be scoring 75%+.

Project 5 — Weather API App (DUE TODAY)

■ **DEADLINE: Weather API App: August 22**

Push your final weather_app.py with all features complete:

```
git add weather_app.py .env.example README.md git commit -m 'feat: project 5 weather API app complete' git push # Note: commit .env.example (with fake values shown) – NEVER commit .env itself
```

AWS Practice Exam #2 (Do This Today)

■ **Take a second full 65-question CLF-C02 practice exam. Target score: 75%+**

- Use a DIFFERENT practice test than last time — different question bank = better coverage
- Time yourself: 90 minutes, no pausing
- After finishing: categorize wrong answers by service (IAM? S3? Lambda?) — study the most-missed service tonight
- Score 75%+: you are on track. Score below: spend the rest of today on AWS study only.

AWS Flashcard Review — Key Services to Know Cold

COMMAND	WHAT IT DOES
S3	Object storage — buckets, objects, storage classes, versioning
EC2	Virtual servers — AMIs, security groups, pricing models
IAM	Identity management — users, groups, roles, policies, MFA
VPC	Virtual network — subnets, route tables, internet/NAT gateway
RDS	Managed relational DB — Multi-AZ (HA), Read Replica (scale reads)
Lambda	Serverless compute — pay per invocation, event-driven
CloudFront	CDN — global content delivery, integrates with S3 + EC2
Route 53	DNS — routing policies: simple, weighted, latency, failover
CloudWatch	Monitoring — metrics, alarms, logs, dashboards
CloudTrail	Audit log — records every API call, who did what when

SUNDAY · August 23 — REST DAY

Review exam results · Continue S3 website · Flash cards

Rest Day tasks (45 min max):

- Review every wrong answer from Practice Exam #2
- Continue working on your S3 static website if not done
- Do 15 minutes of AWS flashcard review
- Look at your GitHub — make sure all 5 projects so far have professional READMEs

DAY 19 · Monday, August 24

S3 Website Polish + EC2 Web Server Start

Finish your cloud deployment. Then launch your first real EC2 server.

S3 Static Website — Final Checklist

- HTML is clean and professional — no broken links, no placeholder text
- Site loads from the S3 website endpoint URL (<http://bucket-name.s3-website-us-east-1.amazonaws.com>)
- All images load correctly from S3
- README.md includes the live URL and deployment steps
- Committed to GitHub with: **git commit -m 'feat: project 6 S3 static website live'**

START Project 7 — EC2 Web Server (Due Monday Aug 31)

■ **DEADLINE: EC2 Web Server — COMPLETE by Monday August 31**

Launch an EC2 instance and host a web page from a real Linux server:

- In AWS Console → EC2 → Launch Instance: Amazon Linux 2023 AMI, t2.micro (Free Tier), create a new key pair (download the .pem file!)
- Security Group: allow SSH (port 22) from your IP only, allow HTTP (port 80) from anywhere
- SSH into your instance: **chmod 400 your-key.pem** then **ssh -i your-key.pem ec2-user@YOUR_PUBLIC_IP**
- Install nginx: **sudo dnf install nginx -y** then **sudo systemctl start nginx** then **sudo systemctl enable nginx**
- Visit http://YOUR_PUBLIC_IP in a browser — you should see the nginx welcome page

■ *Getting your first EC2 instance running is a big milestone. Screenshot it when the nginx page loads.*

AWS Study — Final Review (30 min)

7 days until the exam. Start your final review cycle. Today: re-read your notes on **IAM + S3 + EC2**.

DAY 20 · Tuesday, August 25

EC2 Web Server — Custom Content + User Data

Replace the nginx default page with your own. Automate server setup with User Data.

Lesson: Deploying Content to EC2

```
# After SSH-ing in: replace default nginx page with your own HTML sudo nano
/usr/share/nginx/html/index.html # Paste your HTML (your portfolio page from the S3 project
works great) # Ctrl+X to save # Verify nginx is serving it: curl http://localhost # from inside
the instance # Then visit http://YOUR_PUBLIC_IP in browser # --- USER DATA SCRIPT (automates
all setup on launch) --- # In EC2 Launch → Advanced → User Data, paste this: #!/bin/bash dnf
update -y dnf install nginx -y systemctl start nginx systemctl enable nginx echo '<h1>Hello
from EC2 – Miryam Feleke</h1>' > /usr/share/nginx/html/index.html # This script runs
automatically when the instance launches. # Launch a NEW instance with this User Data – verify
it auto-serves your page. # --- CHECKING LOGS --- sudo tail -f /var/log/nginx/access.log # see
who's visiting sudo tail -f /var/log/nginx/error.log # see errors sudo systemctl status nginx #
check if nginx is running
```

Lab — Day 20: Deploy Custom HTML to EC2

- Replace the default nginx page with your portfolio HTML
- Verify it loads from your browser via `http://YOUR_PUBLIC_IP`
- Terminate that instance, launch a new one with a User Data script that auto-installs nginx and your HTML
- Verify the new instance serves your page without any manual SSH commands
- Take a screenshot of your portfolio page loading from EC2

AWS Study — Final Review: VPC + Lambda + CloudWatch (30 min)

5 days until the exam. Today: re-read your notes on VPC, Lambda, and CloudWatch.

- Draw a VPC diagram from memory: public subnet (EC2 + Internet Gateway), private subnet (RDS), NAT Gateway
- Explain Lambda pricing to yourself out loud
- Name 3 CloudWatch use cases

DAY 21 · Wednesday, August 26

AWS Practice Exam #3 + EC2 Polish

Three practice exams before the real one. You should be at 80%+ by now.

AWS Practice Exam #3 (Morning — Do First)

■ Take a third full 65-question practice exam. Target score: 80%+

- 65 questions, 90-minute timer, no pausing, no notes
- After: sort wrong answers by category and identify any remaining weak spots
- Study those specific weak spots for 45 minutes immediately after reviewing the exam
- Score 80%+: you are ready. Score below: spend all of tomorrow on AWS review, skip coding today.

EC2 Web Server — Complete These Before Friday

- EC2 instance running with nginx serving your custom HTML page via public IP
- User Data script automates the full setup on launch
- Security group allows HTTP (80) from anywhere and SSH (22) from your IP only
- Document the public IP and architecture in a README.md
- Screenshot of the page loading in browser attached or linked in README

AWS Study — Pricing and Support Plans (20 min)

- **AWS Support Plans:** Basic (free), Developer (\$29/mo), Business (\$100/mo+), Enterprise (\$15,000/mo+)
- Business plan = access to Cloud Support Engineers, Trusted Advisor full checks
- **Trusted Advisor:** checks cost optimization, performance, security, fault tolerance, service limits
- **AWS Pricing Calculator:** estimate costs before deploying — calculator.aws

DAY 22 · Thursday, August 27

Full AWS Review Day — Exam is Tomorrow

No new topics today. Review, memorise, drill. The exam is in two days.

Complete This Review Checklist Today

TOPIC	CAN YOU DO IT? (Yes / Not Yet / Need Review)
Explain the Shared Responsibility Model in 60 seconds	
Name the 6 pillars of the Well-Architected Framework	
Describe IAM: users, groups, roles, policies, MFA, least privilege	
Explain S3 storage classes and when to use each	
Describe EC2 pricing models: On-Demand, Reserved, Spot, Dedicated	
Explain VPC: public/private subnets, Internet Gateway, NAT Gateway	
Describe RDS Multi-AZ (availability) vs Read Replica (performance)	
Explain Lambda: triggers, pricing, use cases vs EC2	
Describe CloudFront: what it does, what an origin is, edge locations	
Explain Route 53 routing policies: Simple, Weighted, Latency, Failover	
Describe CloudWatch: metrics, alarms, logs, dashboards	
Explain CloudTrail: what it logs, why it matters for auditing	
Name the AWS Support plans and what Business tier gets you	
Describe AWS Organizations and Service Control Policies	
Explain the 6 benefits of cloud computing	

Today's Study Plan

Morning (2 hrs): Work through the checklist above — for each item, say the answer out loud or write it. If you hesitate, add it to a "review again" list.

Afternoon (1 hr): Do 30 targeted practice questions from your weakest topics.

Evening (30 min): Read the AWS CLF-C02 exam guide PDF from [aws.amazon.com](https://aws.amazon.com/exam/guide/) — know the domain breakdown.

■ Go to bed early. Exam is tomorrow (or Monday — depending on your schedule).

DAY 23 · Friday, August 28

S3 Static Website DUE + EC2 Final Push

Ship your S3 website. Finalize EC2. Rest before the exam.

Project 6 — S3 Static Website (DUE TODAY)

■ **DEADLINE: S3 Static Website: August 28**

Final checklist:

- Portfolio page loading at S3 website endpoint URL
- GitHub repo has all HTML/CSS files committed
- README.md includes the live URL
- `git commit -m 'feat: project 6 S3 static website complete' → git push`

EC2 Web Server — Final Polish

- Your custom HTML page serves from EC2 public IP
- User Data script documented and committed to GitHub
- **IMPORTANT: Stop (not terminate) your EC2 instance when not in use to avoid charges**
- README.md explains: instance type, AMI, security group config, how to launch and access

Light Review Only Today (30 min max)

You have done the deep study. Today is light review and rest:

- Re-read your one-page cheat sheet (if you made one)
- Do 10 practice questions — just to stay warm, not to stress
- Sleep 8 hours tonight. A rested brain outperforms a cramming brain.

DAY 24 · Saturday, August 29

EC2 Web Server DUE + Final Exam Prep

Project 7 ships today. Light study only. You are ready.

Project 7 — EC2 Web Server (DUE TODAY)

■ **DEADLINE: EC2 Web Server: August 29**

Final requirements:

- EC2 instance (Amazon Linux, t2.micro) with nginx serving your custom HTML via public IP
- Security group: HTTP 80 open, SSH 22 restricted to your IP
- User Data script that automates full setup on instance launch
- README.md with: screenshot, public IP (or note that it changes on restart), step-by-step launch instructions

```
git add README.md userdata.sh git commit -m 'feat: project 7 EC2 web server complete' git push
# Also STOP your EC2 instance in the console to avoid charges
```

Final GitHub Portfolio Review

Before the exam: open your GitHub profile. Verify you have:

- cloud-engineering-labs/ — all 9 daily labs (day1 through day9)
- bmi_calculator.py + README
- password_generator.py + README
- expense_tracker.py + README
- contact_book.py + README
- weather_app.py + README (+ .env.example)
- index.html (S3 site) + README with live URL
- userdata.sh / README for EC2 project
- .gitignore at repo root

Today: Rest + Light Review Only

- Read through your notes one final time — slowly, don't cram
- Do 15 practice questions at most
- Prepare your exam logistics: know where you are taking the exam (online or test center), have your ID ready
- Drink water, eat well, sleep early

SUNDAY · August 30 — REST + FINAL REVIEW

Final light review · Rest · Prepare for exam day

Today (30 min max, then REST):

- Review the CLF-C02 domain weights: Cloud Concepts 24%, Security & Compliance 30%, Cloud Technology 34%, Billing 12%
- Read through your AWS notes one final time
- Do NOT take a full practice exam today — rest your brain
- Confirm your exam booking, make sure your webcam/ID is ready if doing online proctoring

DAY 25 · Monday, August 31

EXAM DAY — AWS Cloud Practitioner CLF-C02

This is what the whole month was for. Go get certified.

Morning Routine Before the Exam

- Wake up at your normal time — do not oversleep
- Eat a real breakfast
- Review your top 10 trickiest concepts one last time — 15 minutes only
- Be at your test center (or have your online proctoring ready) 15 minutes early

Exam Strategy

- 65 questions, 90 minutes — that is 83 seconds per question. Do not spend more than 2 minutes on any one question.
- Mark difficult questions and come back — do not let one hard question derail your pacing
- Eliminate wrong answers first — usually 2 of the 4 options are clearly wrong
- Trust your preparation — if you have been scoring 78%+ on practice exams, you will pass
- Passing score: 700/1000 — that is roughly 70%

Most Tested Topics — Read These Right Before Walking In

COMMAND	WHAT IT DOES
Shared Responsibility	AWS = hardware/infrastructure. You = data, OS, application, IAM.
IAM	Users, Groups, Roles, Policies. Never use root. Least privilege always.
S3	Object storage. Standard vs Glacier. Versioning. Bucket policies. Static hosting.
EC2	Virtual machines. On-Demand vs Reserved vs Spot. Security groups. AMIs.
RDS	Multi-AZ = high availability (failover). Read Replica = scale reads.
Lambda	Serverless. Pay per invocation. Event-driven. No server management.
VPC	Private network. Public/private subnets. Internet Gateway. NAT Gateway.
CloudWatch	Monitoring: metrics, alarms, logs. Not to be confused with CloudTrail.

CloudTrail	Audit log of all API calls. Who did what and when.
Well-Arch	6 pillars: Excellence, Security, Reliability, Performance, Cost, Sustainability.

After the Exam

You get a preliminary pass/fail on screen immediately after submitting.

- If you PASS: screenshot the result. Celebrate. Post on LinkedIn. Module 3 starts tomorrow.
- If you need to retake: review weak spots, schedule a retake for next week, do not dwell — the SAA is next.

■ **DEADLINE: AWS Cloud Practitioner exam: August 31, 2026**

MODULE 2 COMPLETE — Review + Mock Interview

Consolidate Everything. Practice Explaining Your Work Out Loud.

Do this after the exam — same day or next day.

Module 2 Review Checklist

TOPIC	CAN YOU DO IT? (Yes / Not Yet / Need Review)
Read from and write to a file using <code>open()</code> with the <code>with</code> statement	
Use <code>try/except/finally</code> to handle errors on any user input or file operation	
Parse JSON from a string and from a file using <code>json.loads()</code> / <code>json.load()</code>	
Write a list comprehension and a dictionary comprehension	
Use <code>os.path.exists()</code> , <code>os.makedirs()</code> , <code>os.environ.get()</code>	
Call a REST API with <code>requests.get()</code> , parse the JSON response	
Define a class with <code>__init__</code> , methods, and <code>__str__</code>	
Use inheritance: write a subclass that calls <code>super().__init__()</code>	
Explain IAM: users, groups, roles, policies, least privilege	
Explain S3: buckets, objects, storage classes, static hosting	
Explain EC2: instance types, security groups, pricing models	
Explain VPC: subnets, Internet Gateway, NAT Gateway	
Explain the difference between RDS Multi-AZ and Read Replica	
Explain Lambda: what it is, when to use it instead of EC2	
Describe CloudWatch vs CloudTrail	
Name the 6 Well-Architected pillars in order	

Module 2 Mock Interview — Answer Out Loud

Q: Explain the AWS shared responsibility model.

A: AWS is responsible for the security OF the cloud — the physical hardware, data centers, networking, and hypervisor. Customers are responsible for security IN the cloud — their data, operating systems, network configurations, IAM policies, and application code. A simple way to remember it: AWS locks the building, you lock your room.

Q: What is IAM and why is the principle of least privilege important?

A: IAM — Identity and Access Management — controls who can access AWS resources and what they can do. Least privilege means granting only the permissions someone needs to do their specific job, and nothing more. If credentials are ever compromised, the attacker can only do what that credential was allowed to do. I enforced this in my projects by creating a dedicated IAM user with only the permissions my app needed, never using root credentials in code.

Q: What is the difference between S3 Standard and S3 Glacier?

A: S3 Standard is for frequently accessed data — it has millisecond retrieval, high durability, and high throughput. S3 Glacier is for archival storage — it is much cheaper per GB but retrieval takes minutes to hours. You choose based on how often you need the data. I used S3 Standard for my static portfolio website because users need it instantly, and Glacier would make sense for old log archives.

Q: What is a VPC and why would you use one?

A: A Virtual Private Cloud is a logically isolated section of AWS where you define your own network topology: IP address ranges, subnets, route tables, and security rules. You use a VPC to control what resources can communicate with each other and what is exposed to the internet. For my EC2 web server project, I put it in a public subnet so HTTP traffic could reach it, while in production a database would go in a private subnet with no direct internet access.

Q: Why would you use Lambda instead of EC2?

A: Lambda is serverless — you write the function, AWS handles all the infrastructure. You only pay for the milliseconds your code actually runs, and it scales automatically to handle any number of requests. EC2 is a virtual machine you provision and manage yourself — better for long-running processes, stateful apps, or workloads that need specific OS configurations. Lambda is ideal for event-driven workloads: process a file uploaded to S3, respond to an API Gateway request, or run a scheduled job.

Q: How do you handle an exception in Python? Walk me through an example from your projects.

A: I use a try/except block. In my Weather API app, I wrapped every API call: `try: r = requests.get(url, timeout=10); r.raise_for_status(); data = r.json()` — then except `requests.exceptions.HTTPError`: if the status was 404 I raised a `ValueError` telling the user the city was not found; for other HTTP errors I re-raised. I also had `except requests.exceptions.ConnectionError` to handle no internet, and finally I used `raise_for_status()` consistently so I never silently swallowed a bad response.

Q: Walk me through your Weather API App project.

A: I built a command-line Python app that fetches current weather and a 5-day forecast for any city using the OpenWeatherMap API. The user enters a city, and the app returns temperature in Celsius and Fahrenheit, humidity, wind speed, and a daily forecast breakdown. I stored the API key in a `.env` file using `python-dotenv` — never in the code or GitHub. I used the `requests` library and wrapped every API call in try/except to handle city-not-found errors, timeouts, and connection failures. The search history is saved to a JSON file. The code is on my GitHub with a README explaining how to set up the API key and run the app.

What's Coming in Module 3 — Preview

Starting September 1, you enter Module 3: AWS Solutions Architect Associate + GlamCloud AI.

- AWS: deep dives into VPC, Route 53, ALB/NLB, Auto Scaling, DynamoDB, CloudFront, SQS/SNS, ECS
- Projects: Serverless Image Uploader (Lambda + API Gateway + S3), Cloud Resume Challenge
- GlamCloud AI: initialize your flagship portfolio project repository
- BIG deadline: AWS SAA exam — September 30, 2026
- Take one full SAA practice exam every Saturday throughout September

Module 2 complete. You have a Cloud Practitioner certification, five Python projects, two live cloud deployments, and a GitHub portfolio that proves you can ship real code to real infrastructure.